Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Traffic Signal System**.

In this article, we'll guide you through a clear 9-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- **The system should control traffic signals** at a single intersection (4 traffic lights as a unit).
- **The system should manage automatic cycling** through phases (NORTH → EAST → SOUTH → WEST).
- **The system should handle emergency vehicle priority requests** by pausing the automatic cycle.
- **During emergency:** All signals turn RED, emergency direction gets GREEN, then resume cycle from pause.
- **The system should track vehicle count** at each approach.
- **The system should prevent conflicting signals** from being active simultaneously.
- **The system should have configurable signal durations** (RED, YELLOW, GREEN) for each direction.
- **The system should allow dynamic adjustment** of signal durations based on traffic conditions.

***Interview Tip*:** *Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.*

Non-Functional Requirements

- The system should respond to user actions within reasonable time.
- The system should have proper state transitions.
- The system should be scalable to multiple intersections.

Edge Cases to Consider

- Emergency vehicle request during signal change.
- Invalid signal state transitions (handled by State Pattern).
- Cycle pause/resume during emergency.

Step 2: Identify Code Entities

1. Intersection (Core Entity)

- id: int [PK]
- name: String
- trafficLights: TrafficLight[] (4 lights: NORTH, SOUTH, EAST, WEST)
- isEmergencyMode: boolean
- emergencyDirection: Direction (nullable)
- isCyclePaused: boolean

2. IntersectionCycle

- intersectionId: int [FK]
- currentPhase: int (0=NORTH, 1=EAST, 2=SOUTH, 3=WEST)
- isPaused: boolean
- pausedAtPhase: int
- phaseStartTime: long (timestamp)

3. TrafficLight

- direction: Enum (NORTH, SOUTH, EAST, WEST)
- currentState: TrafficLightState (State Pattern implementation)
- Consider the following transitions in TrafficLight:
    - **Valid Transitions:** RED → GREEN → YELLOW → RED
    - **Invalid Transitions:** RED → YELLOW, GREEN → RED (blocked by state pattern)

4. SignalTiming

- intersectionId: int [FK]
- direction: Enum (NORTH, SOUTH, EAST, WEST)
- greenDuration: int (seconds)
- isDynamic: boolean (for traffic-based adjustment)
- yellowDuration is a constant (3 seconds) for safety

5. VehicleCounter

- direction: Enum (NORTH, SOUTH, EAST, WEST)
- count: int
- lastUpdate: long (timestamp)

6. EmergencyRequest

- id: int [PK]
- intersectionId: int [FK]
- direction: Enum (NORTH, SOUTH, EAST, WEST)
- duration: int (seconds)
- isActive: boolean

7. TrafficLightState

- State interface for traffic light state management

8. RedState

- Concrete state for RED traffic light

9. GreenState

- Concrete state for GREEN traffic light

10. YellowState

- Concrete state for YELLOW traffic light

11. OffState

- Concrete state for OFF traffic light

***Interview Tip*:** *Clearly defining entities with their attributes early in the design helps you map relationships, enforce rules (like valid state transitions), and implement patterns like State Pattern more effectively.*

Step 3: Visualize Interaction Flows

1. Intersection Management Flows:

- **Intersection Creation Flow:**
  Create intersection → Initialize 4 traffic lights → Set default signal timings → Start automatic cycle
- **Intersection Status Flow:**
  Request status → Return all signal states, cycle info, and current timings

---

2. Automatic Cycle Management Flows:

- **Normal Cycle Flow:**
    - Cycle through phases: NORTH → EAST → SOUTH → WEST
    - Each phase: GREEN (configurable duration) → YELLOW (configurable duration) → RED → Next phase
    - State Pattern ensures valid transitions: RED → GREEN → YELLOW → RED
- **Cycle Pause/Resume Flow:**
  Pause cycle → Remember current phase → Resume from same phase

---

3. Signal Timing Management Flows:

- **Timing Configuration Flow:**
  Set signal timing → Update SignalTiming for direction → Apply to next cycle
- **Dynamic Timing Adjustment Flow:**
  Traffic condition detected → Calculate optimal timing → Update SignalTiming → Apply immediately or next cycle

---

4. Emergency Management Flows:

- **Emergency Request Flow:**
  Emergency request → PAUSE cycle → ALL signals transition to RED (following proper state sequence) → Emergency direction GREEN → Wait duration → Resume cycle from pause

- **Emergency End Flow:**
  End emergency → All signals transition to RED (following proper state sequence) → Resume cycle from paused phase

---

5. Vehicle Counting Flows:

- **Count Update Flow:**
  Vehicle detected → Update count for direction → Trigger dynamic timing adjustment if enabled (in future)
- **Count Query Flow:**
  Request count → Return vehicle count for direction

---

6. State Transition Flows:

- **Valid State Transition Flow:**
  TrafficLight.turnGreen() → currentState.turnGreen(this) → setState(new GreenState())
- **Invalid State Transition Flow:**
  TrafficLight.turnYellow() → currentState.turnYellow(this) → throws InvalidStateTransitionException
- **Emergency State Transition Flow:**
  Emergency transition → Check current state → Follow proper sequence (GREEN → YELLOW → RED) → Handle each state appropriately → Log transition sequence

---

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Service Layer → Domain Layer

Each layer has a distinct role:

- **Client/UI Layer:** Provides the interface for end users or external systems to interact with the traffic signal system (e.g., dashboards, mobile apps, APIs).
- **Controller Layer:** Acts as the entry point for requests, orchestrates calls to services, and converts input/output between the UI and business logic.
- **Service Layer:** Contains the business logic, rules, and workflows for managing intersections, cycles, emergencies, timings, and state transitions.
- **Domain Layer:** Defines the core entities, states, and relationships that model the real-world traffic signal system, independent of infrastructure or UI.

CONTROLLERS:

1. IntersectionController (Main Controller)

- - void createIntersection(int id, String name)
- - Intersection getIntersection(int intersectionId)
- - void startCycle(int intersectionId)
- - void displayStatus(int intersectionId)

2. EmergencyController (Emergency Management)

- - void requestEmergency(int intersectionId, Enum direction, int duration)
- - void endEmergency(int intersectionId)

3. TrafficController

- - void updateVehicleCount(Enum direction, int count)
- - int getVehicleCount(Enum direction)

4. TimingController (Timing Management)

- - void setSignalTiming(int intersectionId, Enum direction, int greenDuration)
- - void enableDynamicTiming(int intersectionId, Enum direction, boolean enable)
- - SignalTiming getSignalTiming(int intersectionId, Enum direction)

SERVICES:

1. IntersectionService (Core Service)

- - void createIntersection(int id, String name)
- - Intersection getIntersection(int intersectionId)
- - void startAutomaticCycle(int intersectionId)
- - void pauseCycle(int intersectionId)
- - void resumeCycle(int intersectionId)
- - IntersectionCycle getCycle(int intersectionId)
- - void setAllSignalsToRed(int intersectionId)
- - void emergencySetAllSignalsToRed(int intersectionId)
- - void setSignalToGreen(int intersectionId, Direction direction)
- - void setSignalToYellow(int intersectionId, Direction direction)
- - void setSignalToRed(int intersectionId, Direction direction)
- - void setSignalToOff(int intersectionId, Direction direction)

2. EmergencyService (Core Emergency Service)

- - void requestEmergency(int intersectionId, Enum direction, int duration)
- - void endEmergency(int intersectionId)
- - EmergencyRequest getActiveEmergency(int intersectionId)

3. TrafficService

- - void updateVehicleCount(Enum direction, int count)
- - int getVehicleCount(Enum direction)

4. TimingService (Timing Management)

- - void setSignalTiming(int intersectionId, Enum direction, int greenDuration)
- - void enableDynamicTiming(int intersectionId, Enum direction, boolean enable)
- - SignalTiming getSignalTiming(int intersectionId, Enum direction)
- - void adjustTimingBasedOnTraffic(int intersectionId, Enum direction)
- - int calculateOptimalGreenDuration(int vehicleCount)

REPOSITORIES:

1. IntersectionRepository

- - void save(Intersection intersection)
- - Intersection findById(int intersectionId)
- - void updateCycle(int intersectionId, IntersectionCycle cycle)
- - void updateEmergencyMode(int intersectionId, boolean emergencyMode, Enum direction)

2. EmergencyRepository

- - void save(EmergencyRequest request)
- - EmergencyRequest getActiveEmergency(int intersectionId)

- - void updateStatus(int requestId, boolean isActive)

3. TrafficRepository

- - void updateCount(Enum direction, int count)
- - int getCount(Enum direction)

4. TimingRepository (Timing Data Access)

- - void saveSignalTiming(SignalTiming timing)
- - SignalTiming getSignalTiming(int intersectionId, Enum direction)
- - void updateSignalTiming(int intersectionId, Enum direction, int greenDuration)

STATE PATTERN IMPLEMENTATION:

1. TrafficLightState (Interface)

- + void turnGreen(TrafficLight trafficLight)
- + void turnYellow(TrafficLight trafficLight)
- + void turnRed(TrafficLight trafficLight)
- + void turnOff(TrafficLight trafficLight)
- + String getStateName()
- + boolean canTransitionTo(TrafficLightState newState)

2. RedState (Concrete State)

- + void turnGreen(TrafficLight trafficLight) // Valid transition
- + void turnYellow(TrafficLight trafficLight) // Invalid - throws exception
- + void turnRed(TrafficLight trafficLight) // No change
- + void turnOff(TrafficLight trafficLight) // Valid transition
- + String getStateName() // Returns "RED"

3. GreenState (Concrete State)

- + void turnGreen(TrafficLight trafficLight) // No change
- + void turnYellow(TrafficLight trafficLight) // Valid transition
- + void turnRed(TrafficLight trafficLight) // Invalid - throws exception
- + void turnOff(TrafficLight trafficLight) // Valid transition
- + String getStateName() // Returns "GREEN"

4. YellowState (Concrete State)

- + void turnGreen(TrafficLight trafficLight) // Invalid - throws exception
- + void turnYellow(TrafficLight trafficLight) // No change
- + void turnRed(TrafficLight trafficLight) // Valid transition
- + void turnOff(TrafficLight trafficLight) // Valid transition
- + String getStateName() // Returns "YELLOW"

5. OffState (Concrete State)

- + void turnGreen(TrafficLight trafficLight) // Valid transition
- + void turnYellow(TrafficLight trafficLight) // Valid transition
- + void turnRed(TrafficLight trafficLight) // Valid transition
- + void turnOff(TrafficLight trafficLight) // No change
- + String getStateName() // Returns "OFF"

6. TrafficLight (Context - Uses State Pattern)

- - direction: Direction
- - currentState: TrafficLightState
- + void setState(TrafficLightState newState)
- + void turnGreen()
- + void turnYellow()
- + void turnRed()
- + void turnOff()
- + String getCurrentStateName()
- + boolean canTransitionTo(TrafficLightState newState)

7. Intersection (Enhanced with Emergency Methods)

- - id: int

- - name: String
- - trafficLights: Map<Direction, TrafficLight>
- - isEmergencyMode: boolean
- - emergencyDirection: Direction
- - isCyclePaused: boolean
- + void setAllSignalsToRed() // Enhanced with proper state transitions
- + void emergencyTransitionToRed(Direction direction) // Emergency transition method
- + void setSignalToGreen(Direction direction)
- + void setSignalToYellow(Direction direction)
- + void setSignalToRed(Direction direction)
- + void setSignalToOff(Direction direction)

STEP-5: CORE USE CASES & METHODS

1. IntersectionController Use Cases:

- **Intersection Creation Use Case:**
  createIntersection() → IntersectionService.createIntersection() → IntersectionRepository.save() → Intersection created with 4 traffic lights and default timings
- **Intersection Status Use Case:**
  getIntersection() → IntersectionService.getIntersection() → IntersectionRepository.findById() → Intersection with all traffic light states and timings returned
- **Automatic Cycle Use Case:**
  startCycle() → IntersectionService.startAutomaticCycle() → Timer schedules cycle with configurable durations → Automatic cycling begins
- **Cycle Pause/Resume Use Case:**
  EmergencyService.requestEmergency() → IntersectionService.pauseCycle() → Cycle paused at current phase

→ EmergencyService.endEmergency() → IntersectionService.resumeCycle() → Cycle resumes from paused phase

- **Emergency Request Use Case:**
  requestEmergency() → EmergencyService.requestEmergency() → IntersectionService.pauseCycle() → IntersectionService.emergencySetAllSignalsToRed() → Emergency direction GREEN → Timer for resume

---

2. EmergencyController Use Cases:

- **Emergency Request Use Case:**
  requestEmergency() → EmergencyService.requestEmergency() → IntersectionService.pauseCycle() → IntersectionService.emergencySetAllSignalsToRed() → Emergency direction GREEN → Timer for resume
- **Emergency End Use Case:**
  endEmergency() → EmergencyService.endEmergency() → IntersectionService.emergencySetAllSignalsToRed() → IntersectionService.resumeCycle() → Cycle resumes from paused state

---

3. TrafficController Use Cases:

- **Vehicle Count Update Use Case:**
  updateVehicleCount() → TrafficService.updateVehicleCount() → TrafficRepository.updateCount() → Count updated → Trigger dynamic timing adjustment if enabled

- **Vehicle Count Query Use Case:**
  getVehicleCount() → TrafficService.getVehicleCount() → TrafficRepository.getCount() → Count returned
- **Dynamic Timing Trigger Use Case:**
  updateVehicleCount() → TrafficService.updateVehicleCount() → TrafficRepository.updateCount() → TimingService.adjustTimingBasedOnTraffic() → TimingRepository.updateSignalTiming() → Dynamic timing applied

---

4. TimingController Use Cases:

- **Signal Timing Configuration Use Case:**
  setSignalTiming() → TimingService.setSignalTiming() → TimingRepository.updateSignalTiming() → Signal timing updated for direction
- **Dynamic Timing Adjustment Use Case:**
  adjustTimingBasedOnTraffic() → TimingService.adjustTimingBasedOnTraffic() → Calculate optimal duration → Update timing → Apply to next cycle
- **Dynamic Timing Enable/Disable Use Case:**
  enableDynamicTiming() → TimingService.enableDynamicTiming() → TimingRepository.updateSignalTiming() → Dynamic timing enabled/disabled for direction

---

5. State Pattern Use Cases:

- **Valid State Transition Use Case:**
  TrafficLight.turnGreen() → currentState.turnGreen(this) → setState(new GreenState()) → State changed successfully

- **Invalid State Transition Use Case:**
  TrafficLight.turnYellow() → currentState.turnYellow(this) → throws InvalidStateTransitionException → Transition blocked
- **State Query Use Case:**
  TrafficLight.getCurrentStateName() → currentState.getStateName() → Returns current state name
- **Emergency State Transition Use Case:**
  emergencyTransitionToRed() → Check current state → GREEN → YELLOW → RED → YELLOW → RED → RED → (no change) → Log transition sequence

---

Step 6: Apply OOP Principles & Design Patterns

The system design leverages key design patterns and OOP principles to ensure robust state handling, modular structure, and easy scalability across intersections.

Design Patterns Used:

- Repository Pattern: Provides abstraction for all data access operations.
- Service Layer Pattern: Separates and centralizes business logic.
- State Pattern: Manages traffic light states and validates transitions.

OOP Principles Applied:

- Single Responsibility Principle: Each class is focused on a single well-defined purpose.
- Open/Closed Principle: The system can be extended with new intersections or features without modifying existing code.

- Encapsulation: Domain objects protect and manage their own data and behavior internally.
- Dependency Inversion Principle: High-level services rely on repository interfaces rather than concrete implementations.

Interview Tip: *Emphasize how the State Pattern streamlines traffic light control logic and how the combination of repository and service layers ensures maintainability and scalability as the system grows.*

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Emergency vehicle request during signal change:** PAUSE the automatic cycle and handle emergency immediately
- **Invalid signal state transitions:** State Pattern enforces valid transitions (Red → Green → Yellow → Red) with emergency override
- **Cycle pause/resume during emergency:** Maintain exact pause state and resume from same phase
- **Dynamic timing adjustment during active cycle:** Apply timing changes to next cycle, not current
- **Traffic-based timing conflicts:** Validate timing changes within safe ranges (min/max durations)
- **State transition exceptions:** Handle `InvalidStateTransitionException` gracefully with logging

---

Implementation Strategies:

- **Cycle Pause/Resume:** `IntersectionCycle` maintains pause state and resumes from exact phase
- **Safety Validation:** State Pattern enforces valid signal transitions (Red → Green → Yellow → Red) with emergency sequence handling
- **Emergency Scheduling:** Timer handles automatic emergency duration and resume
- **Timing Validation:** TimingService validates timing changes within safe ranges
- **Dynamic Adjustment:** Apply timing changes to next cycle to avoid mid-cycle disruptions
- **State Exception Handling:** Catch InvalidStateTransitionException and log for debugging

---

Class Diagram